Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Parallel Programming
Assignment 13: Consensus
Spring Semester 2026

Overview

Consensus is a fundamental problem in parallel / distributed computing with many practical applications. Abstractly, the consensus problem is about agreeing on the same value among multiple agents, while allowing for agents to fail.

The consensus problem for N threads can be phrased as follows: Each thread proposes a value. After a finite number of steps each thread must agree on the same value, which must also be equivalent to one of the proposed values. Since we allow threads to fail, the protocol must be wait-free (we cannot wait for another threads actions, since that thread might have failed and thus we could never make progress). Each thread has a unique integer id between 0 and N-1.

Consensus is one of the pillars of distributed cryptocurrencies such as bitcoin: All ”participants” need to agree on who possesses how much money at some point in time. However, for distributed cryptocurrencies to work in the presence of malicious participants we cannot assume they always cooperate.

Exercise 1 – Consensus Properties

A consensus protocol must fulfil three properties: consistency (the return value is the same for all threads), validity (the result is equal to one of the values proposed), and wait-freedom (each thread must finish in a finite number of steps, independent of other threads).

Below we show **incorrect** implementations of a consensus protocol in pseudocode. Which property does each snippet violate when used with two threads?

```
int decide(int proposed, int thread_id) {
  return proposed;
}
-----
```

```
int decide(int proposed, int thread_id) {
  return 0;
}
----
```

```
int shared = 0; //shared atomic register
int decide(int proposed, int thread_id) {
  if (shared == 0) {
    shared = proposed;
```

```
  }
  return shared;
}
-----

int shared = 0; //shared atomic register
synchronized int decide(int proposed, int thread_id) {
  if (shared == 0) {
    shared = proposed;
  }
  return shared;
}
```

Exercise 2 – Consensus Hierarchy

a) Some objects are unable to implement a valid consensus protocol for some number of threads. For example, we have shown you a proof for the impossibility of consensus for two threads using only atomic registers in the exercise session. However, a (wait-free) FIFO queue with the operations enqueue() and dequeue() can implement wait-free consensus among two threads. Implement a consensus protocol for two threads using a FIFO queue, assume enqueue and dequeue are wait-free atomic operations, writing pseudocode is sufficient here, since Java does not implement such a FIFO anyway. Hint: You can assume the FIFO queue is initialized in any way that helps.

We call the maximum number of threads for which an object (such as a FIFO queue) can implement consensus the *consensus number* of that object.

b) Show that a FIFO queue with the wait-free and atomic operations enqueue, dequeue, and peek (returns the same value dequeue would return, but does not modify the queue) has consensus number ∞.

Exercise 3 – Binary Consensus

In the proof for the impossibility of consensus for two threads with atomic registers we will use a simplified version of consensus, where the proposed values could only be zero or one. Show that binary consensus is equivalent to consensus for two threads.

One way to show equivalence would be to show that we can implement one from the other (in both directions) using only atomic registers.